

Sistema Multi-Agente para Analisis de Segmentacion

Documentacion Tecnica

Andres Fernando Gomez Rojas | Junio 2026

Arquitectura del Sistema

El sistema utiliza un patron **supervisor con fan-out** implementado con LangGraph. El supervisor analiza cada consulta del usuario, la descompone en sub-tareas, y las asigna a uno o varios agentes especialistas que se ejecutan en paralelo. Un nodo synthesizer combina los resultados en una respuesta coherente.

A diferencia de un router simple que envia cada consulta a un solo agente, este orquestador puede invocar **multiples agentes por consulta**. Por ejemplo, 'compara los clusters 0 y 3, valida estadisticamente y recomienda acciones' activa los tres agentes simultaneamente.

```
Query del usuario
|
[SUPERVISOR] -- clasifica, descompone, planifica
|
|--- analyst (perfila, compara, drivers)
|--- statistician (tests, validacion)
|--- strategist (marketing, recomendaciones)
|
[SYNTHESIZER] -- combina en respuesta unica
|
Respuesta al usuario + trace en MLflow
```

Stack tecnologico

Componente	Tecnologia	Justificacion
Grafo de agentes	LangGraph	Control explicito del flujo, fan-out nativo, estado tipado
Integracion LLM	LangChain + Anthropic	Tool use, structured output, bind_tools
Modelo LLM	Claude Sonnet 4	Balance costo/calidad, buen tool calling
Trazabilidad	MLflow 3.x	Autolog LangChain, traces, model registry
API	FastAPI	Async, OpenAPI auto, Pydantic validation
Frontend	React + Nginx	SPA con chat, clusters, traces
Infraestructura	Docker Compose + AWS EC2	3 servicios: api, mlflow, frontend

Agentes Especialistas

Agente	Personalidad	Tools	Cuando se activa
Analyst	Riguroso, factual, cita numeros. Usa z-scores para contextualizar.	get_cluster_overview, get_cluster_profile, compare_clusters, rank_cluster_drivers	"describe C2", "compara C0 y C3", "cuales son los clusters"
Statistician	Cauteloso, valida con tests. Reporta effect size ademas de p-value.	test_significance, count_visitors, get_top_categories	"es significativa la diferencia?", "cuantos tienen bounce > 0.5?"
Strategist	Orientado a negocio. Basa recomendaciones en reglas codificadas.	suggest_marketing_strategy, get_cluster_profile, get_cluster_overview	"que estrategia para C3?", "como llegar a mobile users?"

El supervisor no es un simple router. Tiene cuatro responsabilidades: clasificacion de scope (rechaza preguntas fuera de dominio), descomposicion de consultas multi-tema, planificacion con paralelizacion cuando las sub-tareas son independientes, y sintesis final de respuestas heterogeneas.

Catalogo de Tools

Cada tool es una funcion Python decorada con `@tool` de LangChain. El LLM lee el docstring para decidir cual invocar. Las tools estan organizadas en 4 capas:

Capa	Tool	Que hace
Exploracion	get_cluster_overview	Tabla resumen de los 4 clusters: nombre, tamano, metricas clave
Exploracion	get_cluster_profile	Perfil completo de un cluster: medias con z-score, composicion categorica
Exploracion	compare_clusters	Comparacion de 2 clusters ordenada por effect size (Cohen d)
Exploracion	rank_cluster_drivers	Variables que mas distinguen un cluster del resto, por z-score
Estadistica	test_significance	Mann-Whitney U entre 2 clusters: p-value + rank-biserial r
Busqueda	count_visitors	Cuenta visitantes con filtros opcionales (pandas query)
Busqueda	get_top_categories	Distribucion categorica de un cluster en una dimension
Estrategia	suggest_marketing_strategy	Recomendaciones pre-codificadas: canal, timing, mensaje, KPIs

Principios de diseno

Tools atomicas y composables: una tool por capacidad, no una por pregunta. El LLM las combina segun necesite. Por ejemplo, para responder 'compara C0 y C3 en bounce' el analyst llama `compare_clusters` + `get_cluster_profile` x2.

Estrategias pre-codificadas: `suggest_marketing_strategy` retorna reglas de negocio definidas en codigo, no generadas por el LLM. Esto evita hallucinations en recomendaciones de marketing.

Human-in-the-loop: el sistema analiza y recomienda, pero nunca ejecuta acciones con efecto en produccion. Un humano decide si implementar las recomendaciones.

Trazabilidad con MLflow

Cada consulta al sistema genera un trace completo en MLflow que incluye:

Que se registra	Donde verlo	Para que sirve
Query del usuario	Parametros del run	Reproducibilidad
Agentes invocados	Tags del run	Entender el routing del supervisor
Tool calls con args y resultados	Traces (arbol de ejecucion)	Debugear por que el agente respondio algo
Mensajes LLM (input/output)	Traces (spans)	Auditar lo que el LLM vio vs lo que respondio
Latencia por nodo	Metricas del run	Optimizar cuellos de botella
Costo estimado	Traces	Control de presupuesto de API
Modelo registrado	Model Registry	Versionamiento del agente

El autolog de LangChain (**mlflow.langchain.autolog**) captura automaticamente el arbol de ejecucion completo: supervisor → agentes en paralelo → tool calls → synthesizer, sin necesidad de instrumentar manualmente cada paso.

Despliegue

Puerto	Servicio	URL
3000	React Frontend (chat, clusters, traces)	http://3.16.212.12:3000
8000	FastAPI (6 endpoints REST + Swagger)	http://3.16.212.12:8000/docs
5000	MLflow (trazabilidad, model registry)	http://3.16.212.12:5000

API Endpoints

Metodo	Ruta	Descripcion
GET	/health	Estado del sistema: datos cargados, API key, modelo
GET	/clusters	Overview de los 4 clusters
GET	/clusters/{id}	Perfil detallado de un cluster
GET	/clusters/{id}/strategy	Estrategia de marketing para un cluster
POST	/agent/query	Consulta al sistema multi-agente
GET	/traces	Historial de ejecuciones (MLflow)

Ejemplo de consulta multi-agente

```
curl -X POST http://3.16.212.12:8000/agent/query \  
-H 'Content-Type: application/json' \  
-d '{"query": "Compara C0 y C3, valida estadisticamente  
y recomienda acciones de marketing"}'
```

Resultado: supervisor descompone en 3 sub-tareas,
activa analyst + statistician + strategist en paralelo,
synthesizer combina en respuesta unica.
11 tools invocadas, ~\$0.13, ~1.8 minutos.